uc3m

ARCOS
COMPUTER ARCHITECTURE, COMMUNICATIONS AND SYSTEMS RESEARCH GROUP

Preguntas frecuentes

Práctica 2 – Sistemas operativos

(Go to English version tab)

Errores comunes detectados en la Práctica 1 (Tomarlo en cuenta para la Práctica 2):

*** Nombre incorrecto del archivo ZIP.** *** NIA(s) incorrectos o mal indicados.** *** Formato incorrecto en `autores.txt`.** *** No ejecutar el script `checker_ssso_p1.py` antes de entregar.** *** No probar el programa en Guernika.** *** Entregar zips duplicados; varios envíos por equipo.**	*** En la memoria, aseguraos de incluir todos los apartados solicitados exactamente como se especifica. Además, por favor, preparad una memoria clara y de buena calidad.** *** No utilizar el Makefile para compilar (`make`).** *** No utilizar el formato de salida solicitado.** *** No utilizar el código de error correcto como `-1`**

En caso de dudas no resueltas en este documento, contacta con el profesor de prácticas de tu grupo reducido:

Número de grupo reducido	Nombre	Correo
80	Javier Rodríguez	jrines@inf.uc3m.es
81	Juan Carlos Cano	jucanor@pa.uc3m.es
84, 85 y 801	Genaro Sánchez	gesanche@pa.uc3m.es
87 y 141	Catherine Torres	cattorre@inf.uc3m.es
88 y 89	Paula Sánchez	paulasan@inf.uc3m.es

Preguntas generales

1. **¿Puedo modificar el código proporcionado?**

Sí, tienen la libertad de modificar el código proporcionado, incluida la función procesar_linea. Pueden adaptar el código a sus necesidades.

uc3mshell

1. **En el enunciado se indica: “Deben leer carácter a carácter hasta encontrar un salto de línea e insertar un ‘\0’ al final de la cadena de caracteres”. ¿Esto significa que no podemos utilizar un buffer para leer varios caracteres y luego procesarlos, con el objetivo de reducir el número de llamadas al sistema? Es decir, ¿debemos hacer una llamada al sistema por cada carácter leído, o se permite leer en un buffer y después procesarlo carácter a carácter dentro del programa?**

Tienen la libertad de aplicar la implementación que les parezca más adecuada. Siempre y cuando su implementación cumpla las normas, por ejemplo, no deben utilizar *fread*.

2. **¿Puedo utilizar la función popen para ejecutar comandos?**

No, en la práctica se requiere que el estudiante utilice específicamente la llamada al sistema exec.

3. **¿Se deben controlar las redirecciones adicionales como un error o simplemente implementar su ejecución , es decir, sigo haciendo las entradas, etc?**
   Por ejemplo:

```shell
comando1 > a.txt | comando2 > b.txt | comando3 > c.txt
```

Para el alcance de la práctica no se pueden tener varias redirecciones del mismo tipo en una línea. Se deben considerar como un formato incorrecto.

4. **¿Qué diferencia hay entre un error de comando y un error de system call?**

Es importante distinguir:

Error en un comando (NO termina la shell)

Si exec se ejecuta correctamente pero el comando falla, la shell debe continuar.
Ejemplo:

```
char *args[] = { "ls", "dir", NULL };
execvp(args[0], args);
```

Aquí `ls` se ejecuta correctamente, pero falla porque `dir` no existe.
La shell continúa.

Error en una system call (SÍ termina la shell)

Si `exec` falla, significa que no se pudo ejecutar ningún programa.
Ejemplo:

```
char *args[] = { "comando_que_no_existe", NULL };
execvp(args[0], args);
perror("exec failed");
```

Aquí `execvp` devuelve error (-1).
La shell debe terminar.

5. **Al hacer *exit* en la terminal, ¿debo implementar mi propia llamada/comando *exit* para finalizar la ejecución del programa o del proceso que esté ejecutando?**

No, para hacer exit no se pide que se implemente vuestro propio programa o comando *exit*. Simplemente interceptad la llamada, realizad el manejo necesario y utilizad la llamada exit de la librería de C.

6. **La ejecución en background, ¿Es solo para los comandos dentro del programa uc3mshell.c? La cuestión es que si ejecutamos un comando como sleep 100 & y ya no hay más comandos después en el fichero de entrada, el programa uc3mshell debería acabar con el exit, pero el problema es que el proceso que se ha quedado ejecutando sleep 100 pasaría a ser un proceso orphan. Así que si queremos evitarlo, tendríamos que hacer un wait bloqueante antes de finalizar el programa uc3mshell.c, lo cual iría en contra de la ejecución en modo background.**

La ejecución en background es para todos los comandos que contienen el símbolo & al final de la línea.
El escenario tal como lo planteas es correcto. Sin embargo, mi recomendación es colocar el wait bloqueante antes de finalizar el programa uc3mshell, ya que así evitarás que el proceso se quede en zombie en caso de que existan más comandos por ejecutar que puedan tomarle más tiempo al programa.
En este sentido, no estarías yendo en contra de la ejecución en modo background porque sleep sí lo habrías ejecutado sin esperar por él, mientras ejecutas el resto de los comandos. Si no hay más comandos no habrías realizado un "wait" explícito para ese proceso.

7. **En las redirecciones tenemos que hacer el control de errores de que la de entrada solo puede estar en el primer comando, la de salida en el último... Y en el background que solo puede estar en el último comando?**

Sí, deben comprobar que la sintaxis se respete.
Te recomiendo primero enfocarte en que las redirecciones y el background funcionen con ejemplos válidos como los que les compartimos y ya después pasar al control de errores.

Mycalc

1. **¿mycalc debe poder ejecutarse también utilizando pipes?**

No se espera que lo metáis en pipes. Sin embargo, queda a decisión de cada uno hacerlo.

2. **¿mycalc debe incorporar obligatoriamente la función de historial?**

No es obligatorio, no forma parte de la implementación solicitada para esta segunda práctica.

3. **Al compilar el código (make) se borra o no se genera el mycalc y solo puedo llamarlo a través de uc3mshell, ¿es eso correcto o debería generar un programa como en la práctica anterior?**

Sí, el programa mycalc en esta práctica se comporta como un mandato interno, por lo que cuando se compila uc3mshell el programa mycalc no debe “existir”. Para utilizarlo debéis llamarlo como comando de vuestro script. Por ejemplo:

```shell
## Uc3mshell P2
mycalc 6 + 7
```

4. **En el caso de mycalc, ¿Debemos también implementar redirecciones de entrada? En principio, como hacemos una llamada a la función, pasamos los operadores y el operando como argumentos, de modo que no hacemos uso de funciones tales como fgets o scanf lo que nos impide hacer la redirección de entrada. Un ejemplo de la situación que planteamos sería el siguiente:**
   echo 2 + 2 > datos_originales.txt
   mycalc < datos_originales.txt > datos_trabajo.txt

Este caso quedaría a decisión suya cómo lo implementen, ya que como comentas los operadores y el operando se pasan como argumentos. Por lo tanto para el siguiente comando:
mycalc 1 + 1 < autores.txt > resultado-mycalc.txt
no veo problema si simplemente ignoran la redirección de entrada o si deciden controlarla el programa tampoco haría nada diferente porque mycalc no utiliza la entrada estándar.
Tu ejemplo "mycalc < datos_originales.txt > datos_trabajo.txt" debería dar un error al ejecutar mycalc porque no sigue la sintaxis solicitada.

Mycp

Exit

1. **¿El comando exit debe ejecutarse correctamente incluso cuando el programa se esté utilizando con pipes?**

No es necesario. El comando “exit” se probará como un comando simple. Sin embargo, queda a decisión de cada uno hacerlo.

Echo

1. **Estamos implementando el caso especial de las comillas en el echo. No sabemos si tenemos que quitarlas solo en el caso de que haya dos, es decir, al principio y al final, o si por ejemplo ponen "Hola mundo , dejamos la primera porque la quieren imprimir.**

Tendrían que quitar las comillas en ambos casos.
En la shell de Linux el comportamiento es un poco distinto. Si solo pones una comilla al inicio o al final la terminal se queda escuchando (bloqueada) esperando a que ingreses la siguiente comilla. Esto lo hace por si quieres trabajar una cadena de texto que incluya saltos de línea. Una vez que ingresas la segunda comilla se imprime el mensaje sin ellas pero con un salto de línea adicional.
Para el alcance de esta práctica, al ejecutar:
echo "Hola mundo"
echo Hola mundo"
echo "Hola mundo
se obtendrá el mismo resultado:
Hola mundo